| | |
|---|---|
| **ДИСЦИПЛИНА** | Инструменты девопс |
| **ИНСТИТУТ** | ИПТИП |
| **КАФЕДРА** | Индустриального программирования |
| **ВИД УЧЕБНОГО МАТЕРИАЛА** | Методические указания к практическим занятиям |
| **ПРЕПОДАВАТЕЛЬ** | Гиматдинов Дамир Маратович |
| **СЕМЕСТР** | 5 семестр, 2026/2027 учебный год |

Ссылка на материал: https://gitverse.ru/dgimatdinov/DevOps_2026/content/master/DEV_9.md

---

# ПРАКТИЧЕСКАЯ РАБОТА № 9

**Тема: Статический деплой и первый CI. GitHub Pages, GitHub Actions, Pylint**

| | |
|---|---|
| **Дисциплина** | DevOps-практикум |
| **Курс** | 3 |
| **Занятие** | 9 из 16 |
| **Блок** | Автоматизация конфигураций и CI/CD (занятие 1 из 4) |
| **Форма работы** | Индивидуальная |
| **Продолжительность аудиторной части** | До 90 минут |

Работа выполняется в новом публичном репозитории GitHub и не требует виртуальной машины, Docker или локальной установки Python. Сборка документации и проверка кода выполняются на GitHub-hosted runner. Подготовка к занятию ограничивается созданием репозитория и проверкой доступа по Git.

---

## 1 Цель и задачи работы

Цель работы — сформировать практические навыки автоматической проверки изменений и публикации статической документации средствами GitHub Actions и GitHub Pages.

Задачи:

а) различить события, задания, шаги и действия в GitHub Actions;

б) настроить запуск статического анализатора Pylint при событиях `push` в `main` и `pull_request` в `main`;

в) подключить обязательную проверку к Pull Request и исследовать её поведение при ошибке в коде;

г) собирать сайт документации из Markdown средствами MkDocs и публиковать его на GitHub Pages через артефакт;

д) применить принцип минимально необходимых прав для `GITHUB_TOKEN`;

е) диагностировать отказ проверки и отказ публикации по журналам, а также убедиться, что неудачный выпуск не заменяет предыдущую работающую версию сайта.

**Связь с последующими работами.** В практической работе № 10 конвейер расширяется этапами тестирования, сборки Docker-образа и публикации в реестр. В контрольной работе № 3 добавляется автоматический запуск Ansible и развёртывание на VM. Настоящая работа вводит необходимые для этого механизмы: триггеры, коды возврата, обязательные проверки, права токена, артефакты и окружение деплоя.

**Требование безопасности.** Сайт GitHub Pages в публичном репозитории доступен всем. Публиковать в нём пароли, закрытые ключи, токены, содержимое `.env`, реальные адреса внутренней инфраструктуры и полный файл `docs/server-setup.md` запрещается. В работе используется искусственная учебная документация.

---

# 2 Теоретическая часть

## 2.1 CI и CD

Непрерывная интеграция (CI) предполагает автоматическую проверку каждого предлагаемого изменения. Непрерывная доставка или развёртывание (CD) переносит проверенный результат в целевую среду. Эти процессы связаны, но не тождественны.

В настоящей работе два независимых контура:

1. `CI — code quality` запускается для `push` в `main` и для Pull Request в `main`; он выполняет Pylint и ничего не публикует.
2. `Publish documentation` запускается только после изменения документации в `main`; он собирает и публикует сайт.

Поэтому зелёная проверка Pylint означает лишь соответствие Python-кода заданным правилам. Она не доказывает корректность ссылок в документации, работоспособность приложения или успешность будущего деплоя. Этот пробел обнаруживается в задании 4 и устраняется в самостоятельной работе.

## 2.2 Модель GitHub Actions

Workflow описывается YAML-файлом в каталоге `.github/workflows/`. Минимальная иерархия приведена в таблице 1.

Таблица 1. Основные элементы GitHub Actions

| Элемент | Назначение в настоящей работе |
|---|---|
| `on` | События запуска: `push`, `pull_request`, ручной `workflow_dispatch` |
| `permissions` | Права автоматически выданного `GITHUB_TOKEN` |
| `job` | Независимая единица выполнения на runner |
| `runs-on` | Образ runner; используется `ubuntu-latest` |

| Элемент | Назначение в настоящей работе |
|---|---|
| `step` | Команда `run` или готовое действие `uses` |
| `environment` | Окружение `github-pages`, в котором фиксируется публикация и её URL |
| `concurrency` | Запрет одновременных публикаций одного сайта |

GitHub-hosted runner — временная виртуальная машина. После завершения задания её состояние не сохраняется. Следовательно, зависимости устанавливаются заново, а результат, необходимый другому заданию, передаётся через артефакт. Кэш зависимостей ускоряет повторные запуски, но не заменяет файл требований и не является хранилищем результата сборки.

Команда шага завершается кодом возврата. Нулевой код означает успех, ненулевой — отказ шага и, при отсутствии специальных исключений, всего задания. Параметр `continue-on-error: true` в работе не используется: он превратил бы обнаруженную ошибку в ложно успешную проверку.

## 2.3 События `push` и `pull_request`

Событие `push` возникает после появления новых коммитов в ветке. Событие `pull_request` относится к предлагаемому изменению и позволяет проверить результат до слияния. Фильтр `branches: [main]` для `pull_request` ограничивает целевую ветку, а не исходную.

Публикация в работе привязана только к `push` в `main`. Это исключает ситуацию, при которой любой ещё не рассмотренный Pull Request заменяет общедоступный сайт. Ручной триггер `workflow_dispatch` предусмотрен для повторной публикации без создания пустого коммита.

Фильтр `paths` у workflow публикации исключает лишнюю сборку при изменении только Python-кода. В обязательных проверках фильтры следует применять осторожно: пропущенная из-за фильтра required check может оставаться в состоянии ожидания и блокировать PR.

Обязательная проверка действует не только на кнопку merge. Пока для нового коммита не получен успешный результат required check, GitHub отклоняет и непосредственный `git push` в защищённую ветку (`GH013: Repository rule violations found` для ruleset, `GH006: Protected branch update failed` для классической защиты). Поэтому после включения правила `protect-main` в задании 2 все изменения, включая изменения workflow, вносятся только через ветку и Pull Request.

## 2.4 Pylint и статический анализ

Pylint исследует Python-код без его обычного выполнения: выявляет ошибки, неиспользуемые переменные и импорты, нарушения соглашений и потенциальные дефекты проектирования. Код возврата Pylint ненулевой при обнаружении сообщений включённых категорий, поэтому анализатор естественным образом останавливает шаг CI.

Параметр `--score=no` отключает только итоговую числовую оценку, но не сами диагностические сообщения и не ненулевой код возврата. Отключать правило, обнаружившее реальную ошибку, в задании запрещается: требуется исправить код.

Версии зависимостей зафиксированы в `requirements-dev.txt`. Это делает запуски воспроизводимыми: обновление инструмента не должно незаметно изменить правила проверки посреди курса.

### 2.5 MkDocs, артефакт и GitHub Pages

MkDocs преобразует Markdown-файлы из `docs/` в статический HTML-сайт. Команда:

```
mkdocs build --strict --site-dir site
```

создаёт каталог `site/`. Режим `--strict` превращает предупреждения, включая сообщения о ряде битых внутренних ссылок, в ошибку сборки. Каталог `site/` не хранится в Git: это производный результат, который должен быть воспроизведён из исходников.

Для публикации средствами GitHub Actions в настройках Pages выбирается источник **GitHub Actions**, а не ветка. Workflow выполняет четыре операции:

1. собирает сайт;
2. подготавливает Pages действием `actions/configure-pages`;
3. упаковывает именно содержимое `site/` действием `actions/upload-pages-artifact`;
4. публикует артефакт действием `actions/deploy-pages` в окружение `github-pages`.

Файл `index.html` должен находиться на верхнем уровне артефакта. Загрузка корня всего репозитория вместо каталога `site/` является ошибкой: исходные Markdown-файлы и служебные файлы не заменяют собранный сайт.

Если сборка новой версии завершилась ошибкой, задание `deploy` не запускается. Ранее опубликованная версия при этом сохраняется. Failed deployment не равнозначен автоматическому откату уже начавшейся публикации, но в сценарии настоящей работы отказ происходит до этапа `deploy`.

### 2.6 Права токена и параллельные деплои

GitHub автоматически создаёт для запуска `GITHUB_TOKEN`. Workflow запрашивает только необходимые права:

| Право | Назначение |
|---|---|
| `contents: read` | Прочитать содержимое репозитория для checkout |
| `pages: write` | Создать публикацию GitHub Pages |
| `id-token: write` | Получить OIDC-токен, необходимый действию `deploy-pages` для подтверждения публикации |

Права проверяются разными действиями и на разных этапах. `configure-pages` и `deploy-pages` обращаются к API Pages, поэтому им нужно `pages: write`. Кроме того, `deploy-pages` перед созданием публикации запрашивает OIDC-токен; без `id-token: write` переменные для его получения на runner не создаются, и шаг завершается ошибкой ещё до обращения к Pages. Одного `pages: write` поэтому недостаточно.

CI-проверке достаточно `contents: read`. Персональный токен доступа и пароль GitHub в workflow не используются.

Два одновременных деплоя могут завершиться не в порядке создания коммитов. Группа `concurrency: pages` упорядочивает публикации. Значение `cancel-in-progress: false` не обрывает уже начавшийся деплой; ожидается актуальный следующий запуск. Для CI допустима иная политика, но её следует выбирать осознанно.

---

# 3 Подготовка к занятию

## 3.1 Создание репозитория

Выполняется до занятия или в первые пять минут при устойчивом доступе к GitHub.

1. Войдите в GitHub с учётной записью, настроенной в практических работах № 1–3.
2. Создайте новый **публичный** репозиторий `devops-docs` с файлом `README.md`. Не добавляйте лицензию и `.gitignore`: они будут созданы в работе.
3. Склонируйте репозиторий по SSH и проверьте состояние:

```
git clone git@github.com:<login>/devops-docs.git
cd devops-docs
git switch main
git status
git remote -v
```

Здесь и далее `<login>` заменяется фактическим именем учётной записи без угловых скобок. Все команды выполняются на хостовой системе, если отдельно не указано «на GitHub».

**Почему новый репозиторий.** Это исключает случайную публикацию серверной документации и локальных секретов из предыдущих работ. Для бесплатного личного аккаунта Pages доступен в публичном репозитории; использовать приватный репозиторий в рамках данной работы не следует.

## 3.2 Проверка готовности

Таблица 2. Условия готовности к аудиторной части

| Номер | Условие | Подтверждение |
|---|---|---|
| 1 | Создан публичный репозиторий `devops-docs` | Страница репозитория открывается без авторизации в приватном окне |
| 2 | Репозиторий склонирован, рабочая копия чиста | `git status` |
| 3 | Настроены имя и адрес электронной почты автора коммитов | `git config user.name`, `git config user.email` |
| 4 | Push по SSH выполняется без пароля учётной записи GitHub | `git push` после первого коммита |
| 5 | В браузере доступны вкладки Actions и Settings репозитория | Визуальная проверка |

Локальные Python, Pylint и MkDocs для основной части не требуются. Не расходуйте время занятия на их установку.

---

# 4 Порядок выполнения аудиторной части

Таблица 3. Регламент аудиторного занятия

| Этап | Содержание | Время, мин |
|---|---|---|
| 1 | Проверка готовности | 5 |
| 2 | Задание 1. Исходная документация и код | 15 |
| 3 | Задание 2. CI, required check и инцидент с Pylint | 20 |
| 4 | Задание 3. Публикация Pages и инцидент с правами | 25 |
| 5 | Задание 4. Неудачный выпуск документации и восстановление | 15 |
| 6 | Резерв, контроль результата, ответы на вопросы | 10 |
| | Итого | 90 |

По завершении работы должны существовать два workflow, обязательная проверка `pylint`, успешно опубликованный сайт документации и журналы трёх намеренно воспроизведённых инцидентов. Основной сценарий не предполагает установки пакетов на компьютер студента.

## 4.1 Задание 1. Исходная документация и код

**Шаг 1.1.** В ветке `main` создайте каталоги:

```
mkdir -p docs src .github/workflows
```

Создайте `.gitignore`:

```
site/
__pycache__/
*.pyc
```

Создайте `requirements-dev.txt`:

```
mkdocs==1.6.1
pylint==4.1.3
```

Обе зависимости нужны на runner. Фиксация версий относится к учебной воспроизводимой конфигурации; при дальнейшей эксплуатации обновления выполняются отдельным осознанным изменением с повторной проверкой.

**Шаг 1.2.** Создайте `src/status.py`:

```
"""Формирование строки состояния для учебного репозитория."""


def format_status(version: str, deployed: bool) -> str:
    """Возвращает человекочитаемое состояние публикации."""
    state = "deployed" if deployed else "draft"
    return f"{version}: {state}"


if __name__ == "__main__":
    print(format_status("1.0", deployed=True))
```

Этот небольшой модуль служит объектом статического анализа. Написание прикладной логики не является целью работы.

**Шаг 1.3.** Создайте `mkdocs.yml`, заменив `<login>` и `[ФИО]`:

```
site_name: DevOps Operations Handbook
site_description: Учебная документация по проверке и публикации изменений
site_author: "[ФИО]"
repo_name: <login>/devops-docs
repo_url: https://github.com/<login>/devops-docs

theme:
  name: mkdocs
  locale: ru

nav:
  - Главная: index.md
  - Эксплуатация: runbook.md
  - Журнал инцидентов: incidents.md
```

**Шаг 1.4.** Создайте `docs/index.md`:

```
# DevOps Operations Handbook

Учебный сайт документации, собираемый из Markdown и публикуемый через GitHub
Actions.

**Версия: 1.** Этот маркер используется при исследовании неудачного выпуска.

## Контур поставки

1. Изменение предлагается через Pull Request.
2. Pylint проверяет Python-код.
3. После слияния в `main` MkDocs собирает статический сайт.
```

```
4. GitHub Pages публикует артефакт сборки.

Адрес будущего сайта: `https://<login>.github.io/devops-docs/`.
```

Замените `<login>` и внутри документа. Создайте `docs/runbook.md`:

```
# Эксплуатация

## Штатная публикация

1. Убедитесь, что проверка `pylint` в Pull Request успешна.
2. Выполните слияние в `main`.
3. Откройте Actions → Publish documentation.
4. Дождитесь успеха заданий `build` и `deploy`.
5. Откройте URL из окружения `github-pages` и проверьте главную страницу.

## Ручной перезапуск

Для повторной публикации используйте Actions → Publish documentation → Run
workflow.
Пустой коммит для этой цели не требуется.

## Диагностика

Сначала определите отказавший этап: CI, сборка сайта, загрузка артефакта или
публикация.
Затем читайте журнал конкретного шага. Повторный запуск без анализа причины не
считается устранением инцидента.
```

Создайте `docs/incidents.md`:

```
# Журнал типовых инцидентов

| Инцидент | Наблюдаемый признак | Первичный источник диагностики |
|---|---|---|
| Неиспользуемая переменная | Красная проверка `pylint` в PR | Журнал шага Static analysis |
| Недостаточные права публикации | `build` успешен, `deploy` не получает OIDC-токен | Журнал `deploy-pages` и блок `permissions` |
| Битая ссылка | `mkdocs build --strict` завершается ошибкой | Журнал шага Build documentation |
| Старая версия после успешного деплоя | Зелёный workflow, прежнее содержимое в браузере | URL окружения, время публикации, кэш браузера |

## Правило разбора

В разборе указываются симптом, затронутый компонент, доказательство из журнала, причина, исправление и проверка восстановления. Ответ «перезапустил и заработало» без установления причины неполон.
```

**Шаг 1.5.** Обновите `README.md`:

```
# devops-docs

Учебный репозиторий практической работы № 9: Pylint, MkDocs, GitHub Actions и GitHub Pages.

- исходники документации: `docs/`;
- конфигурация MkDocs: `mkdocs.yml`;
- CI: `.github/workflows/quality.yml`;
- публикация: `.github/workflows/pages.yml`.

Каталог `site/` является результатом сборки и не хранится в Git.
```

**Шаг 1.6.** Зафиксируйте исходное состояние непосредственно в `main`:

```
git add .
git commit -m "docs: add operations handbook skeleton"
git push -u origin main
```

На этом этапе Actions ещё не настроен, поэтому отсутствие запусков ожидаемо. Непосредственный push в `main` используется только для начального наполнения пустого репозитория и для первого workflow CI в шаге 2.1, пока обязательная проверка ещё не включена. После шага 2.3 все изменения выполняются только через PR.

**В отчёт:**

1. Структура репозитория после первого коммита.

2. Ссылка на репозиторий.
3. Краткое объяснение, почему `site/` включён в `.gitignore`.

## 4.2 Задание 2. CI, required check и инцидент с Pylint

### 4.2.1 Первый workflow

**Шаг 2.1.** Оставаясь в `main`, создайте `.github/workflows/quality.yml`:

```yaml
name: CI — code quality

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read

jobs:
  pylint:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v7

      - name: Setup Python
        uses: actions/setup-python@v7
        with:
          python-version: "3.12"
          cache: pip
          cache-dependency-path: requirements-dev.txt

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          python -m pip install -r requirements-dev.txt

      - name: Static analysis
        run: python -m pylint src/status.py --score=no
```

Обратите внимание: `push` отфильтрован по `main`, а `pull_request` — по целевой ветке `main`. Публикации и секретов в этом workflow нет, поэтому токен получает только чтение содержимого.

**Шаг 2.2.** Зафиксируйте workflow:

```
git add .github/workflows/quality.yml
git commit -m "ci: add pylint check for main and pull requests"
git push
```

На GitHub откройте **Actions** → **CI — code quality** → **последний запуск** → **pylint**. Дождитесь зелёного результата. Разверните каждый шаг и найдите:

- событие, вызвавшее запуск;
- версию Python;
- установку зафиксированных версий из `requirements-dev.txt`;
- завершение Pylint без сообщений об ошибках.

Если workflow не появился, прежде всего проверьте путь `.github/workflows/quality.yml`, расширение `.yml`, фактический push в `main` и отступы YAML. Не создавайте дубликаты файла с другими именами.

#### 4.2.2 Обязательная проверка

**Шаг 2.3.** После первого успешного запуска откройте **Settings** → **Rules** → **Rulesets** и создайте правило для ветки:

1. выберите **New branch ruleset**;
2. задайте имя `protect-main`;
3. добавьте в targets ветку по умолчанию `main`;
4. включите **Require status checks to pass**;
5. добавьте обязательную проверку `pylint`;
6. сохраните правило.

Если в используемом интерфейсе применяется классическая защита веток, эквивалент находится в **Settings** → **Branches** → **Add branch protection rule**: Branch name pattern — `main`, затем **Require status checks to pass before merging** и выбор `pylint`.

**Внимание.** Начиная с этого шага прямой `git push` в `main` будет отклоняться: новый коммит ещё не имеет успешной проверки `pylint`, а правило требует её до обновления ветки. Это ожидаемое поведение, а не ошибка настройки. Все последующие изменения, включая workflow публикации и исправления прав, выполняются через отдельную ветку и Pull Request. Список обхода (Bypass list) в правиле не заполняйте: учебная цель — пройти тот же путь, что и любое изменение в командном репозитории.

#### 4.2.3 Инцидент 1: красная проверка в Pull Request

**Шаг 2.4.** Создайте ветку и намеренно внесите дефект:

```
git switch -c fix/status-message
```

Приведите функцию в `src/status.py` к виду:

```
def format_status(version: str, deployed: bool) -> str:
    """Возвращает человекочитаемое состояние публикации."""
    details = "temporary"
    state = "deployed" if deployed else "draft"
    return f"{version}: {state}"
```

Переменная `details` не используется. Зафиксируйте изменение и создайте PR:

```
git add src/status.py
git commit -m "fix: prepare detailed status message"
git push -u origin fix/status-message
```

На странице репозитория нажмите **Compare & pull request**, убедитесь, что базовая ветка — `main`, и создайте PR.

**Шаг 2.5.** Дождитесь отказа `pylint`. В журнале шага **Static analysis** найдите сообщение категории `W0612` (`unused-variable`) и строку исходного файла. На странице PR зафиксируйте красную проверку и состояние, препятствующее обычному слиянию.

**Требуемый разбор:** укажите файл, строку, код и смысл сообщения. Объясните, почему Pylint вернул ненулевой код и почему добавление `continue-on-error: true` было бы не исправлением, а сокрытием дефекта.

**Шаг 2.6.** Удалите строку `details = "temporary"`, не отключая правило Pylint. Отправьте исправление в ту же ветку:

```
git add src/status.py
git commit -m "fix: remove unused temporary variable"
git push
```

PR обновится автоматически, открывать второй PR не нужно. Дождитесь зелёной проверки, выполните merge и удалите удалённую feature-ветку предложенной кнопкой GitHub. Обновите локальную `main`:

```
git switch main
git pull
```

После merge появится отдельный запуск CI по событию `push` в `main`.

**В отчёт:**

1. Успешный первый запуск CI.
2. Настройка required check `pylint`.
3. Красный PR, сообщение `W0612` и объяснение кода возврата.
4. Тот же PR после исправления и успешный merge.

5. Ответ: почему второй `git push` запустил проверку повторно без создания нового PR.

## 4.3 Задание 3. Публикация Pages и инцидент с правами

### 4.3.1 Выбор источника публикации

**Шаг 3.1.** На GitHub откройте **Settings** → **Pages**. В разделе **Build and deployment** → **Source** выберите **GitHub Actions**. Ветку `main` и каталог `/docs` в качестве источника не выбирайте: сборкой и содержимым публикации будет управлять workflow.

### 4.3.2 Workflow с намеренно недостаточными правами

**Шаг 3.2.** Прямой push в `main` уже запрещён правилом `protect-main`, поэтому workflow публикации добавляется через Pull Request. Обновите локальную `main` и создайте ветку:

```
git switch main
git pull
git switch -c ci/publish-docs
```

Создайте `.github/workflows/pages.yml` точно в приведённом виде. В первой версии намеренно отсутствует право `id-token: write`; не исправляйте права до воспроизведения инцидента.

```
name: Publish documentation

on:
  push:
    branches: [main]
    paths:
      - "docs/**"
      - "mkdocs.yml"
      - "requirements-dev.txt"
      - ".github/workflows/pages.yml"
  workflow_dispatch:

# ИНЦИДЕНТ 2: прав для публикации пока недостаточно (нет id-token: write).
permissions:
  contents: read
  pages: write

concurrency:
  group: pages
  cancel-in-progress: false

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
```

```
        uses: actions/checkout@v7

      - name: Setup Python
        uses: actions/setup-python@v7
        with:
          python-version: "3.12"
          cache: pip
          cache-dependency-path: requirements-dev.txt

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          python -m pip install -r requirements-dev.txt

      - name: Build documentation
        run: mkdocs build --strict --site-dir site

      - name: Configure Pages
        uses: actions/configure-pages@v6

      - name: Upload Pages artifact
        uses: actions/upload-pages-artifact@v5
        with:
          path: site

  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}

    steps:
      - name: Deploy to GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v5
```

Связь `needs: build` гарантирует, что публикация начнётся только после успешной сборки и загрузки артефакта. Поле `environment.url` получает адрес сайта из выхода действия публикации.

**Шаг 3.3.** Зафиксируйте workflow, отправьте ветку и создайте PR в `main`:

```
git add .github/workflows/pages.yml
git commit -m "cd: add documentation publishing workflow"
git push -u origin ci/publish-docs
```

В PR выполнится только `pylint`: у `pages.yml` нет триггера `pull_request`, поэтому публикация из PR не запускается. Дождитесь зелёной проверки, выполните merge и удалите ветку. Merge создаёт в `main` коммит, изменяющий `.github/workflows/pages.yml`, что соответствует фильтру `paths` и запускает публикацию.

Откройте **Actions** → **Publish documentation** и выберите запуск, вызванный merge. Ожидается характерное разделение результата:

- `build` — успешно: MkDocs собрал сайт, `configure-pages` получил настройки Pages, артефакт загружен;
- `deploy` — отказ: действию публикации не выдан OIDC-токен.

#### 4.3.3 Инцидент 2: отказ публикации

**Шаг 3.4.** Раскройте журнал `deploy` и найдите сообщение:

```
Ensure GITHUB_TOKEN has permission "id-token: write".
```

Рядом в журнале может присутствовать техническая причина (`Unable to get ACTIONS_ID_TOKEN_REQUEST_URL env variable` или аналогичная). Диагностическое значение имеет отказ авторизации публикации, а не воспроизведение строки дословно.

Ответьте письменно:

1. Почему успех `build` не означает успех публикации?
2. Почему права `pages: write` недостаточно, а `contents: write` для публикации через артефакт не требуется?
3. Почему использование персонального токена вместо исправления `permissions` является избыточным и менее безопасным решением?

**Шаг 3.5.** Создайте ветку исправления:

```
git switch main
git pull
git switch -c fix/pages-permissions
```

Исправьте блок `permissions`:

```
permissions:
  contents: read
  pages: write
  id-token: write
```

Удалите строку «ИНЦИДЕНТ 2», затем отправьте исправление и создайте PR:

```
git add .github/workflows/pages.yml
git commit -m "fix: grant minimum permissions for pages deployment"
git push -u origin fix/pages-permissions
```

После зелёной проверки `pylint` выполните merge. Дождитесь нового запуска **Publish documentation** по событию `push` в `main` и успеха обоих заданий. Не нажимайте **Re-run jobs** у упавшего запуска: повторный запуск использует прежний коммит и прежнюю версию workflow и воспроизведёт тот же отказ.

**Шаг 3.6.** На странице успешного запуска откройте URL окружения `github-pages`. Ожидаемый адрес:

```
https://<login>.github.io/devops-docs/
```

Проверьте главную страницу, страницу «Эксплуатация» и «Журнал инцидентов». На главной должна быть отметка **Версия: 1**. Если сразу получен `404`, проверьте статус последнего `deploy`, источник Pages и URL проекта; после первого успешного деплоя распространение может занять непродолжительное время. Не меняйте случайным образом настройки DNS: собственный домен в работе не используется.

**Шаг 3.7.** В журнале `build` найдите создание `site/index.html`, а в журнале `Upload Pages artifact` — загрузку каталога `site`. Объясните, почему исходный `docs/index.md` нельзя считать файлом, который браузер получает как готовую главную страницу.

**В отчёт:**

1. Настройка Pages с источником GitHub Actions.
2. Результат первого запуска: зелёный `build` и красный `deploy`.
3. Сообщение об отказе авторизации, исправленный блок `permissions` и ссылки на два PR (`ci/publish-docs`, `fix/pages-permissions`).
4. Успешный `deploy`, URL окружения и окно браузера с версией 1.
5. Ответы на вопросы шагов 3.4 и 3.7.

## 4.4 Задание 4. Неудачный выпуск документации и восстановление

В этом задании CI остаётся зелёным, поскольку проверяет только Python-код. Дефект обнаруживается позже — при сборке документации после merge. Это воспроизводит реальный пробел покрытия проверками, а не ошибку GitHub.

### 4.4.1 Инцидент 3: битая ссылка прошла CI

**Шаг 4.1.** Создайте ветку:

```
git switch -c docs/add-recovery-link
```

В `docs/index.md` замените маркер на:

```
**Версия: 2.** Обновление после исправления ссылки.
```

В конец `docs/runbook.md` добавьте ссылку на пока несуществующий файл:

```
## Восстановление

Перед восстановлением изучите [инструкцию по аварийному восстановлению]
(recovery.md).
```

Файл `docs/recovery.md` не создавайте. Зафиксируйте изменения и создайте PR в `main`:

```
git add docs/index.md docs/runbook.md
git commit -m "docs: add recovery runbook link"
git push -u origin docs/add-recovery-link
```

**Шаг 4.2.** Дождитесь результата PR. Ожидается зелёный `pylint`. Зафиксируйте этот факт и объясните, почему он не свидетельствует о корректности документации. Выполните merge.

#### 4.4.2 Отказ сборки после merge

**Шаг 4.3.** Откройте запуск **Publish documentation**, вызванный merge в `main`. Ожидается:

- ошибка на шаге **Build documentation** с сообщением MkDocs о ссылке на отсутствующий `recovery.md`;
- ненулевой код возврата из-за `--strict`;
- отсутствие запуска `deploy`;
- сохранение опубликованной версии 1.

Откройте сайт и убедитесь, что маркер версии 2 не появился. При необходимости обновите страницу с обходом кэша браузера. Не пытайтесь «исправить» инцидент повторным запуском без изменения исходников.

Заполните краткий разбор:

```
Симптом: failed workflow Publish documentation после merge.
Затронутый компонент: сборка MkDocs, задание build.
Доказательство: сообщение о битой ссылке на docs/recovery.md.
Причина: ссылка указывает на несуществующий файл; --strict превращает
предупреждение в ошибку.
Влияние: новая версия не опубликована; предыдущая версия сайта остаётся доступной.
```

#### 4.4.3 Восстановление

**Шаг 4.4.** Обновите `main` и создайте ветку исправления:

```
git switch main
git pull
git switch -c docs/fix-recovery-link
```

В `docs/runbook.md` замените ошибочную ссылку на существующий файл:

```
Перед восстановлением изучите [журнал типовых инцидентов](incidents.md).
```

Зафиксируйте исправление, создайте второй PR и после зелёной required check выполните merge:

```
git add docs/runbook.md
git commit -m "docs: point recovery section to existing incident log"
git push -u origin docs/fix-recovery-link
```

После merge дождитесь успеха `build` и `deploy`. Откройте сайт: главная страница должна содержать **Версия: 2**, а ссылка на странице «Эксплуатация» должна открывать «Журнал инцидентов».

**В отчёт:**

1. Зелёный CI в PR с битой ссылкой и объяснение пробела проверки.
2. Журнал отказа MkDocs после merge.
3. Сайт с прежней версией 1 после failed build.
4. Исправление ссылки, успешный deploy и сайт с версией 2.
5. Заполненный разбор инцидента из шага 4.3.

---

# 5 Задание для самостоятельной работы

## 5.1 Закрытие пробела проверки документации

Необходимо перенести выявление битой ссылки с этапа после merge на этап Pull Request, не добавляя публикацию из PR.

**Шаг 5.1.** Обновите локальную `main`, создайте ветку и дополните `.github/workflows/quality.yml` шагом сразу после **Static analysis**:

```
git switch main
git pull
git switch -c ci/validate-docs
```

```
      - name: Validate documentation
        run: mkdocs build --strict --site-dir /tmp/docs-site
```

Зависимость MkDocs уже установлена из `requirements-dev.txt`. Результат помещается во временный каталог runner и никуда не загружается. Название задания `pylint` не меняйте: иначе потребуется обновить required check в правилах ветки.

Зафиксируйте и отправьте изменение:

```
git add .github/workflows/quality.yml
git commit -m "ci: validate documentation before merge"
git push -u origin ci/validate-docs
```

**Шаг 5.2.** Создайте PR из `ci/validate-docs` в `main`. Убедитесь, что оба шага успешны. После merge повторите контрольный эксперимент с несуществующей ссылкой в отдельной ветке:

1. добавьте в любой Markdown-файл ссылку `(missing-page.md)`;
2. создайте PR;
3. убедитесь, что проверка теперь красная **до merge**;
4. удалите ошибочную ссылку новым коммитом;
5. дождитесь зелёной проверки и выполните merge.

**Шаг 5.3.** Письменно сравните два расположения одной и той же проверки:

| Место проверки | Когда узнаём о дефекте | Меняется ли опубликованный сайт |
|---|---|---|
| Только `pages.yml` после merge | После попадания дефекта в `main` | Нет, если отказ произошёл до `deploy` |
| `quality.yml` в PR и после merge | До слияния | Нет; дефектный PR не должен быть принят |

Объясните, почему добавление сборки документации в CI не делает триггер `pull_request` в `pages.yml` необходимым.

**В отчёт:** изменённый workflow, красный и зелёный запуски контрольного PR, сравнение по таблице.

## 5.2 Маркер версии в артефакте

Задание выполняется по желанию и помогает связать опубликованное содержимое с конкретным коммитом.

В отдельной ветке (например, `ci/build-metadata`) добавьте в `pages.yml` шаг после **Build documentation**, но до загрузки артефакта, и внесите изменение в `main` через PR:

```
      - name: Add build metadata
        run: |
          printf 'commit=%s\n' "$GITHUB_SHA" > site/build.txt
          printf 'run=%s\n' "$GITHUB_RUN_ID" >> site/build.txt
          date -u '+built_at=%Y-%m-%dT%H:%M:%SZ' >> site/build.txt
```

После публикации откройте `https://<login>.github.io/devops-docs/build.txt` и сопоставьте SHA с коммитом, вызвавшим запуск. Объясните, почему такой маркер полезнее времени последнего изменения `index.md` при расследовании «сайт не обновился».

---

## 6 Факультативное задание. Гонка публикаций

Задание выполняется при наличии времени и на оценку не влияет.

1. Подготовьте два небольших изменения главной страницы в двух отдельных PR.
2. После зелёных проверок выполните merge обоих PR с минимальным интервалом либо сразу после merge одного PR инициируйте ручной запуск **Publish documentation → Run workflow**. Прямой push в `main` для ускорения не используйте: он отклоняется правилом `protect-main`.
3. В Actions наблюдайте состояния `in progress` и `pending` в группе `pages`.
4. Убедитесь, что выполняющийся деплой не был отменён, а ожидающий запуск соответствует более новому состоянию.

Ответьте:

а) какой результат мог бы появиться без `concurrency`, если старый запуск завершился бы позже нового;

б) почему `cancel-in-progress: false` выбрано для публикации;

в) чем желательная политика параллелизма для длительных тестов в PR может отличаться от политики деплоя.

Не используйте этот эксперимент как способ безусловного отмены любых запусков: прерывание деплоя в реальной системе требует отдельного анализа состояния целевой среды.

---

## 7 Быстрая диагностика типовых затруднений

Таблица 4. Диагностический минимум

| **Симптом** | **Вероятная причина** | **Что проверить в первую очередь** |
|---|---|---|
| Workflow не появился в Actions | Неверный путь, расширение, ветка или YAML | `.github/workflows/`, push в `main`, отступы, вкладка Actions |
| CI не запустился для PR | Неверный триггер или целевая ветка | `pull_request.branches: [main]`, base ветку PR |
| Required check не выбирается в настройках | Проверка ещё ни разу не выполнялась | Сначала успешный запуск job `pylint` |
| Merge блокируется в ожидании проверки | Required check была отфильтрована или переименована | Фильтры `paths`, имя job, фактический статус в PR |
| `git push` в `main` отклонён с `GH013`/`GH006` и упоминанием | Правило `protect-main` требует успешной проверки | Ветка и PR вместо прямого push |

| Симптом | Вероятная причина | Что проверить в первую очередь |
|---|---|---|
| `pylint` | до обновления ветки | |
| `build` зелёный, `deploy`: `Ensure GITHUB_TOKEN has permission "id-token: write"` | Токену не выдано право на OIDC-токен | Блок `permissions`: `id-token: write` |
| `build` зелёный, `deploy` возвращает 403 | Недостаточны права токена на Pages | `pages: write`, источник Pages, ограничения организации |
| `deploy` ожидает артефакт | Не задана связь заданий или сборка не выполнилась | `needs: build`, успех upload |
| Pages показывает 404 после успеха | Неверный URL, неготов первый деплой, нет `index.html` наверху артефакта | Environment URL, `path: site`, содержимое сборки |
| Pylint зелёный, сайт не собирается | CI проверяет только Python | Журнал MkDocs; добавить docs check в PR |
| После исправления Re-run снова падает | Повторно запущен старый коммит | Новый push с исправлением, SHA запуска |
| Браузер показывает прежнее содержимое при зелёном deploy | Кэш либо проверяется не тот запуск | SHA/URL окружения, `build.txt`, обновление с обходом кэша |

---

## 8 Подготовка к практической работе № 10

а) изучить назначение модульных тестов и отличие тестового прогона от статического анализа;

б) подготовить устный ответ, почему последовательность «линтинг → тесты → сборка образа → публикация в реестр» не должна продолжаться после ненулевого кода возврата предыдущего этапа;

в) повторить понятия Docker image, tag и container registry из практической работы № 7;

г) изучить различие между артефактом GitHub Actions и опубликованным Docker-образом: оба являются результатами конвейера, но имеют разное назначение и жизненный цикл;

д) убедиться, что в репозитории отсутствуют токены и пароли; в следующей работе публикация в GitHub Container Registry будет выполняться с автоматическим `GITHUB_TOKEN` и правом `packages: write`, без персональных токенов и паролей реестра.

---

## 9 Контрольные вопросы

1. Чем различаются workflow, job, step и action? Какие из этих элементов выполняются на отдельном runner?

2. Почему CI запускается и для `push`, и для `pull_request`, а публикация Pages — только для `push` в `main`?
3. На что указывает фильтр `branches: [main]` в событии `pull_request`: на исходную или целевую ветку?
4. Как ненулевой код возврата Pylint влияет на шаг, job и required check? Почему `continue-on-error` несовместим с целью обязательной проверки?
5. Почему второй push в исходную ветку автоматически обновляет проверки существующего PR?
6. Почему зелёный Pylint не гарантировал корректность ссылок MkDocs в задании 4? Как перенести обнаружение такого дефекта на более ранний этап?
7. Какие функции выполняют `configure-pages`, `upload-pages-artifact` и `deploy-pages`? Почему `deploy` требует `needs: build`?
8. Для чего нужны `pages: write` и `id-token: write`? Почему одного `pages: write` недостаточно, а `contents: write` не требуется?
9. Чем `GITHUB_TOKEN` отличается от персонального токена доступа в контексте настоящей работы?
10. Почему артефактом должен быть каталог `site/`, а не корень репозитория и не исходный каталог `docs/`?
11. Почему при failed build прежний сайт остался доступен? Изменился бы ответ, если бы отказ произошёл после частичного применения новой публикации?
12. Какую задачу решают `concurrency` и `cancel-in-progress: false`?
13. Почему повторный **Re-run jobs** не исправляет ошибку в исходниках или правах? Как проверить, какой коммит использует запуск?
14. Почему в публичном репозитории нельзя размещать `.env` и реальную серверную документацию, даже если позже удалить их отдельным коммитом?
15. Чем артефакт сборки отличается от кэша зависимостей? Что из этого необходимо заданию `deploy`?
16. Почему ссылки на сторонние actions в промышленных workflow рекомендуют закреплять на полный SHA коммита, хотя в учебной работе использованы основные версии тегов?
17. Почему после включения обязательной проверки `pylint` прямой `git push` в `main` отклоняется, хотя отдельный запрет push не настраивался? Как в этих условиях вносится исправление workflow?

---

## 10 Требования к содержанию и оформлению отчёта

Отчёт оформляется в формате Microsoft Word (.docx) и содержит титульный лист, цель и задачи работы, краткую схему CI/CD, материалы из блоков «В отчёт», выполненные в собственном репозитории, разборы трёх аудиторных инцидентов, результаты самостоятельной работы, ссылки на репозиторий, сайт и Pull Request, а также выводы.

Для каждого инцидента приводится пять элементов: симптом, доказательство из журнала, причина, исправление и проверка восстановления. Копии экранов должны позволять различить исходный отказ и последующий успешный запуск; желательно сохранить в кадре SHA коммита или номер запуска.

Перед сдачей убедитесь в выполнении условий таблицы 5.

Таблица 5. Условия, проверяемые перед сдачей отчёта

| Номер | Условие | Подтверждение |
|---|---|---|
| 1 | CI запускается для push в `main` и PR в `main` | Журналы двух типов событий |
| 2 | Pylint имеет только `contents: read`, проверка `pylint` обязательна | `quality.yml`, правило `protect-main` |
| 3 | Воспроизведён и исправлен `W0612` без отключения проверки | Красный и зелёный статусы одного PR |
| 4 | Pages использует источник GitHub Actions | Settings → Pages |
| 5 | Воспроизведён отказ deploy из-за прав и выполнено минимально необходимое исправление | Журнал `deploy` с требованием `id-token: write`, блок `permissions` |
| 6 | Сайт собран из `site/` и успешно опубликован | Зелёные `build` и `deploy`, URL сайта |
| 7 | Воспроизведён отказ MkDocs после merge; прежняя версия сохранилась | Failed build и страница версии 1 |
| 8 | Исправление ссылки опубликовало версию 2 | Успешный deploy и страница версии 2 |
| 9 | Сборка документации перенесена в CI для PR | Красный контрольный PR до merge и последующий зелёный |
| 10 | В Git не добавлены `site/`, `.env`, ключи и токены | `git status`, `git ls-files` |
| 11 | Итоговые workflow не содержат намеренно оставленных дефектов | Успешные последние запуски на `main` |
| 12 | После включения `protect-main` все изменения внесены через PR | История PR, отсутствие прямых коммитов в `main` после шага 2.3 |

Итоговый результат защиты: по любому из трёх инцидентов студент должен без подсказки показать отказавший шаг, объяснить причину и продемонстрировать, какое изменение восстановило работу.

---

## Список использованных источников


1. Workflow syntax for GitHub Actions [Электронный ресурс] // GitHub Docs. URL: https://docs.github.com/en/actions/writing-workflows/workflow-syntax-for-github-actions (дата обращения: 04.11.2026).
2. Using custom workflows with GitHub Pages [Электронный ресурс] // GitHub Docs. URL: https://docs.github.com/en/pages/getting-started-with-github-pages/using-custom-workflows-with-github-pages (дата обращения: 04.11.2026).
3. Configuring a publishing source for your GitHub Pages site [Электронный ресурс] // GitHub Docs. URL: https://docs.github.com/en/pages/getting-started-with-github-pages/configuring-a-publishing-source-for-your-github-pages-site (дата обращения: 04.11.2026).
4. Managing rulesets for a repository [Электронный ресурс] // GitHub Docs. URL: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/managing-rulesets-for-a-repository (дата обращения: 04.11.2026).

5. Building and testing Python [Электронный ресурс] // GitHub Docs. URL: https://docs.github.com/en/actions/use-cases-and-examples/building-and-testing/building-and-testing-python (дата обращения: 04.11.2026).
6. Pylint. User Guide [Электронный ресурс]. URL: https://pylint.readthedocs.io/en/latest/user_guide/index.html (дата обращения: 04.11.2026).
7. MkDocs. User Guide [Электронный ресурс]. URL: https://www.mkdocs.org/user-guide/ (дата обращения: 04.11.2026).
8. Secure use reference [Электронный ресурс] // GitHub Docs. URL: https://docs.github.com/en/actions/security-for-github-actions/security-guides/security-hardening-for-github-actions (дата обращения: 04.11.2026).
9. Releases официальных действий GitHub [Электронный ресурс]: actions/checkout, actions/setup-python, actions/configure-pages, actions/upload-pages-artifact, actions/deploy-pages. URL: https://github.com/actions (дата обращения: 04.11.2026).
10. Troubleshooting required status checks [Электронный ресурс] // GitHub Docs. URL: https://docs.github.com/en/pull-requests/how-tos/merge-and-close-pull-requests/troubleshooting-required-status-checks (дата обращения: 04.11.2026).